Fetch Data with AWS Lambda

Melvin green

Introducing Today's Project!

In this project, I demonstrate how to fetch data from a DynamoDB table using a serverless AWS Lambda function. I built this project to strengthen my understanding of serverless architecture, specifically how Lambda functions can be used to efficiently query and retrieve data from DynamoDB, AWS's fully managed NoSQL database service.

Tools and concepts

Services I used include AWS Lambda, IAM Policies, and DynamoDB. Key concepts I learned include Lambda functions, DynamoDB, execution roles, and IAM permission policies.

Project reflection

This project took me approximately one hour to complete. The most challenging part was writing policies, since I am still learning them.

I chose to do this project today because I am learning how to build a tier three app.

Project Setup

To set up this project, I created a DynamoDB table with a partition key of type String. The partition key acts as a unique identifier that determines which physical partition (server) an item is stored on, allowing DynamoDB to distribute data evenly and enable fast, scalable read/write access.

Here's a polished version: "In my DynamoDB table, I added an item using JSON. DynamoDB is schemeless, meaning I can add attributes as needed, and every item in my database can have a different set of attributes.

AWS Lambda

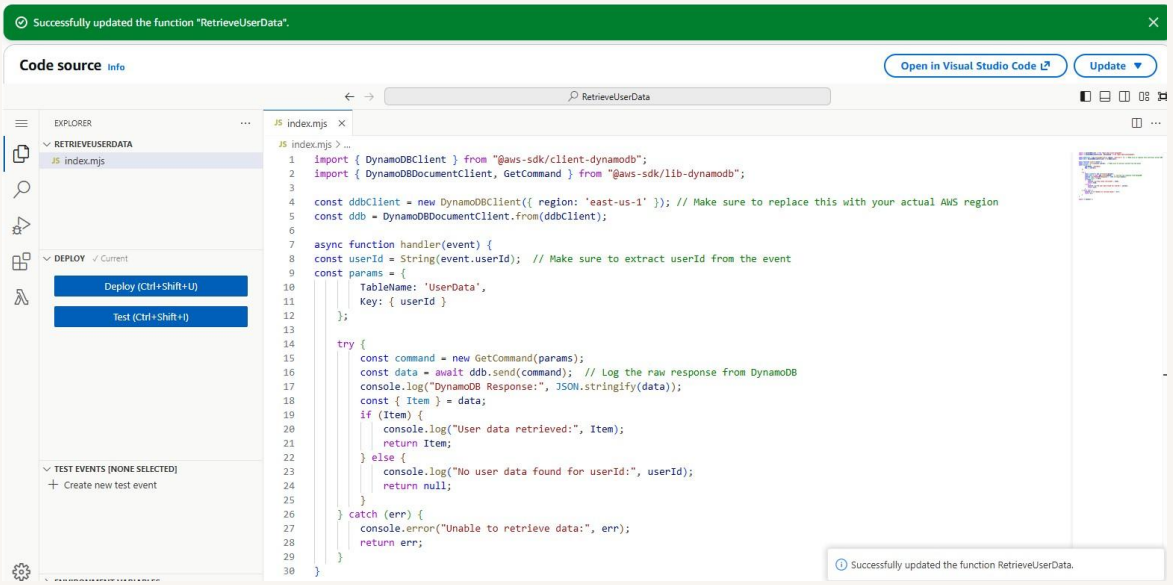
AWS Lambda is a service that lets me run code without needing to manage any computer servers. I'm using Lambda in this project to connect to DynamoDB, allowing the table's items to be retrieved on demand without provisioning or managing a server.

AWS Lambda Function

My Lambda function has an execution role, which is an IAM role assigned to the function. It defines what the function is allowed to do — for example, accessing other AWS services like DynamoDB. By default, the role grants basic permissions for writing logs to CloudWatch.

My Lambda function takes a userId as input, retrieves the corresponding data from the UserData table, and returns it. The second half of the code handles potential errors during the database operations, with error messages indicating what went wrong.

The code uses the AWS SDK, which stands for AWS Software Development Kit. My code uses the SDK, a set of tools that lets developers build applications that interact with AWS services.



Function Testing

To test whether my Lambda function works, I created and ran a test event written in JSON. If the test is successful, I should see the data for userId 1 returned in the response.

The test displayed 'success' because the function itself was able to run without errors. However, the function's actual response indicated 'Access Denied,' because the Lambda function's execution role didn't have permission to access the DynamoDB table.

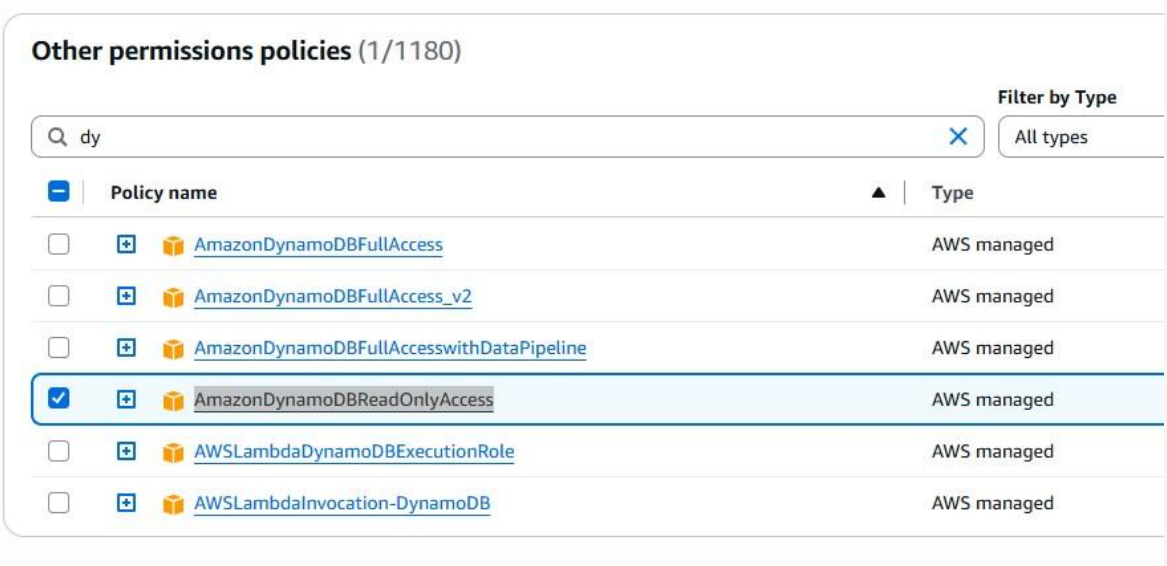
Function Permissions

To resolve the AccessDenied error, I went back to the test results to review the error message because it tells me exactly which permission the Lambda function is lacking.

There were four DynamoDB permission policies to choose from, but I didn't pick AWSLambdaDynamoDBExecutionRole or AWSLambdaInvocation-DynamoDB, because one grants the Lambda function permission to read the DynamoDB Stream, and the other grants permission to respond to an event captured in that stream.

I also didn't pick AmazonDynamoDBFullAccess, because it grants full access to DynamoDB, including creating, deleting, and other write operations.

AmazonDynamoDBReadOnlyAccess was the right choice because it grants permission to only read the data, which is all my Lambda function needs.



Final Testing and Reflection

To validate my new permission settings, I reran the test on my Lambda function. The results were successful, as I was able to see the username, email, and name returned in the response.

Web applications are a popular use case for Lambda and DynamoDB. For example, this pattern could help consumers find products, get product information, or view order history. Another example is social media apps, where Lambda can fetch user profiles or automatically retrieve all linked content. Additionally, Lambda can help news sites or blogs fetch articles based on user queries.

Enhancing Security

For my project extension, I challenged myself to create an inline policy, which allows for more granular control. In this case, I only want my Lambda function to have access to the UserData table.

To create the permission policy, I used JSON because I wanted to get more familiar with writing policies.

When updating a Lambda function's permission policies, there is a risk of accidentally granting access to all of my DynamoDB tables. I validated that my Lambda function still works by running a test to confirm it was able to retrieve the data.

Specify permissions Info

Add permissions by selecting services, actions, resources, and conditions. Build permission statements using the JSON editor.

Policy editor

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Sid": "DynamoDBReadAccess",
            "Effect": "Allow",
            "Action": [
                "dynamodb:GetItem"
            ],
            "Resource": [
                "arn:aws:dynamodb:us-east-1:17975550086:table/UserData"
            ]
        }
    ]
}
```